

FIAP - Faculdade de Informática e Administração Paulista

LINUX SERVICES APPLICATIONS

SISTEMAS OPERACIONAIS

Rafael Caruso

Guilherme Isidoro

João Pedro

ALMALINUX, COMPATIBILIDADE DE ABI E SEGURANÇA

DESENVOLVIMENTO E AUTOMAÇÃO DO VERIFICADOR DE INTEGRIDADE

São Paulo

2026

SUMÁRIO

1. Introdução	3
2. O que é o AlmaLinux	4
3. História e Origem	5
4. AlmaLinux OS Foundation	6
5. RHEL e CentOS Stream	7
6. ABI e a Evolução da Compatibilidade	8
7. Compatibilidade com o RHEL	9
8. Segurança AlmaLinux	10
9. Integridade dos Arquivos	11
10. Verificador de Integridade	12
11. Script Desenvolvido (fim-check.sh)	13
12. Automação e Comparação com AIDE	15
13. Análise e Aplicação Prática	17
14. Conclusão	18
15. Referências	19

RESUMO

Este trabalho analisa a distribuição AlmaLinux OS no contexto do ecossistema Enterprise Linux, destacando sua transição de uma reconstrução exata do Red Hat Enterprise Linux (RHEL) para a atual estratégia de compatibilidade de Interface Binária de Aplicação (ABI). O estudo aborda a criação da AlmaLinux OS Foundation, sua relevância para a continuidade de ambientes corporativos e as diferenças estratégicas em relação ao CentOS Stream. Como aplicação prática de segurança em sistemas operacionais, apresenta-se o desenvolvimento de um verificador de integridade de arquivos em Shell Script, denominado *fim-check.sh*. A ferramenta estabelece uma *baseline* utilizando a função criptográfica SHA-256, monitorando alterações de conteúdo, proprietário, permissões e contexto SELinux em diretórios críticos do sistema, com execução automatizada via *timers* do *systemd*. Conclui-se que o AlmaLinux oferece uma plataforma empresarial compatível e estável, enquanto a implementação de mecanismos de verificação de integridade complementa a arquitetura de proteção, adicionando uma camada essencial de detecção contra alterações não autorizadas.

Palavras-chave: AlmaLinux. Red Hat Enterprise Linux. Segurança de Sistemas. Integridade de Arquivos. Shell Script.

1. INTRODUÇÃO

O Linux possui grande presença em ambientes corporativos por oferecer distribuições voltadas para servidores, aplicações, infraestrutura de redes e serviços críticos. Nesse cenário, as distribuições relacionadas ao ecossistema Red Hat Enterprise Linux (RHEL) possuem importância por oferecerem estabilidade, suporte de longo prazo e recursos para ambientes empresariais. Entre elas está o AlmaLinux OS, desenvolvido como uma alternativa comunitária e gratuita compatível com o ecossistema RHEL.

O AlmaLinux surgiu em um contexto de mudanças no projeto CentOS. Essas mudanças aumentaram a necessidade de alternativas para organizações e usuários que dependiam de uma distribuição estável e próxima do ambiente empresarial do RHEL. Inicialmente, o AlmaLinux buscava compatibilidade binária 1:1 com o RHEL e, posteriormente, modificou sua estratégia para priorizar a compatibilidade de aplicações e da ABI. Atualmente, o projeto é descrito como uma distribuição empresarial de código aberto, gratuita e orientada pela comunidade, com compatibilidade binária com o RHEL.

Este trabalho concentra-se na atividade de Sistemas Operacionais Linux, tendo como temas o AlmaLinux, sua Fundação e a mudança para a compatibilidade de ABI. O segundo eixo está relacionado à segurança do sistema, por meio do desenvolvimento de um verificador de integridade de arquivos denominado fim-check.sh. O briefing da atividade define o script como responsável por estabelecer uma baseline SHA-256 e verificar arquivos e atributos importantes do sistema.

A pesquisa utiliza documentação oficial do AlmaLinux e da Red Hat, além de referências técnicas relacionadas a funções hash e segurança. Na parte prática, são considerados requisitos como execução em modo estrito, utilização de funções, verificação de privilégios e dependências, geração de logs, códigos de saída, idempotência, limpeza de arquivos temporários e validação com ShellCheck.

2. O QUE É O ALMALINUX

O AlmaLinux OS é uma distribuição Linux de código aberto, gratuita e voltada para ambientes empresariais. O projeto é conduzido pela comunidade e possui como característica a compatibilidade binária com o Red Hat Enterprise Linux. Dessa forma, busca permitir que aplicações desenvolvidas para o ambiente RHEL sejam executadas no AlmaLinux sem modificações específicas para a distribuição (ALMALINUX WIKI, 2026).

A distribuição é administrada pela AlmaLinux OS Foundation, organização responsável pela governança e pelo direcionamento do projeto. A Fundação foi criada para estabelecer uma estrutura de governança e evitar que o desenvolvimento dependa exclusivamente de uma única empresa. O projeto conta com membros individuais, patrocinadores e colaboradores, formando uma estrutura comunitária voltada à manutenção de uma plataforma empresarial de longo prazo (ALMALINUX OS FOUNDATION, 2026).

Uma característica importante do AlmaLinux é seu foco em estabilidade. O projeto busca atender ambientes que precisam manter sistemas funcionando por períodos prolongados, oferecendo atualizações de segurança e correções de bugs durante o ciclo de vida das versões principais. Também disponibiliza informações de erratas e recursos relacionados à segurança e conformidade (ALMALINUX WIKI, 2026).

O AlmaLinux possui diferentes repositórios para fornecer os componentes necessários ao sistema. Entre os principais estão BaseOS, AppStream e Extras. O BaseOS contém componentes fundamentais do sistema, enquanto o AppStream fornece aplicações, linguagens de execução e bancos de dados para diferentes cargas de trabalho (ALMALINUX WIKI, 2026).

Assim, o AlmaLinux pode ser entendido como uma alternativa comunitária dentro do ecossistema Enterprise Linux, combinando estabilidade, compatibilidade com RHEL, ciclo de manutenção prolongado e disponibilidade gratuita.

3. HISTÓRIA E ORIGEM DO ALMALINUX

A origem do AlmaLinux está relacionada às mudanças ocorridas no projeto CentOS. Durante muitos anos, o CentOS Linux foi utilizado como alternativa gratuita ao RHEL, principalmente por usuários que buscavam uma plataforma próxima do ambiente empresarial da Red Hat. Por isso, alterações no ciclo de desenvolvimento do CentOS tiveram impacto significativo sobre usuários e organizações.

Com a mudança de direcionamento do CentOS, surgiu a necessidade de novas alternativas. O AlmaLinux foi apresentado como uma distribuição comunitária destinada a ocupar esse espaço. Seu desenvolvimento inicial buscou produzir uma alternativa compatível com o RHEL, utilizando uma abordagem de reconstrução dos componentes do ecossistema Enterprise Linux. A documentação inicial descrevia o AlmaLinux como uma alternativa compatível em nível binário com o RHEL.

O primeiro lançamento estável do AlmaLinux 8 ocorreu em 2021. A versão 8.3 foi apresentada como alternativa ao CentOS e passou a ser considerada por usuários que procuravam uma distribuição empresarial gratuita. Publicações técnicas da época destacaram esse contexto de substituição do CentOS e a proposta de compatibilidade com RHEL (ANDERSON, 2021).

A criação da AlmaLinux OS Foundation estabeleceu uma estrutura de governança para o projeto. Isso permitiu que o desenvolvimento da distribuição não dependesse exclusivamente de uma empresa, com a Fundação assumindo responsabilidades relacionadas à administração do projeto e à organização de sua comunidade.

Nos primeiros anos, o principal objetivo era manter uma relação muito próxima com o RHEL, buscando reproduzir seus componentes de maneira equivalente. Entretanto, mudanças posteriores no acesso às fontes do RHEL tornaram necessário repensar essa estratégia.

Em 2023, a AlmaLinux OS Foundation anunciou uma mudança importante. O projeto deixou de ter como objetivo principal ser uma reconstrução 1:1 do RHEL e passou a priorizar a compatibilidade binária.

4. ALMALINUX OS FOUNDATION

A AlmaLinux OS Foundation é a organização responsável pela governança do projeto AlmaLinux. Sua função é fornecer uma estrutura administrativa para que o desenvolvimento da distribuição seja conduzido de maneira aberta e comunitária. A documentação do projeto apresenta a Fundação como uma organização sem fins lucrativos responsável pela administração e governança do AlmaLinux (ALMALINUX OS FOUNDATION, 2026).

A existência de uma Fundação é importante porque uma distribuição utilizada em ambientes empresariais precisa de continuidade. Empresas que utilizam sistemas operacionais em servidores precisam considerar sua manutenção, atualizações, correções de segurança e capacidade de evolução. Uma estrutura de governança ajuda a estabelecer processos e reduzir a dependência de decisões individuais.

A participação da comunidade também é uma característica importante. O AlmaLinux conta com membros individuais, patrocinadores e empresas colaboradoras. Essa estrutura permite diferentes formas de contribuição para o desenvolvimento, testes, distribuição e manutenção do sistema.

A Fundação também possui mecanismos de governança, incluindo participação de membros e processos de eleição para representantes. Esses processos demonstram a existência de uma estrutura formal para tomada de decisões (ALMALINUX OS FOUNDATION, 2026).

Outro ponto relevante é o equilíbrio entre os interesses da comunidade e as necessidades técnicas de uma distribuição empresarial. Isso envolve decisões sobre compatibilidade, segurança, ciclo de vida, infraestrutura de compilação e distribuição de pacotes.

Essa estrutura tornou-se especialmente importante após a mudança de estratégia de 2023. A decisão de abandonar a reconstrução 1:1 e priorizar a

compatibilidade binária foi também uma decisão estratégica sobre o futuro do projeto diante das mudanças no ecossistema RHEL (ALMALINUX OS, 2023).

5. ALMALINUX, RHEL E CENTOS STREAM

Para compreender o AlmaLinux, é necessário entender sua relação com o RHEL e com o CentOS Stream. O Red Hat Enterprise Linux é uma distribuição empresarial desenvolvida pela Red Hat e utilizada em diferentes ambientes corporativos. O CentOS Stream possui função diferente, atuando como uma distribuição de desenvolvimento contínuo relacionada ao ecossistema do RHEL.

O AlmaLinux surgiu inicialmente com uma proposta próxima da reconstrução do RHEL. A intenção era oferecer uma distribuição compatível em nível binário e reproduzir seus componentes de maneira muito próxima.

A partir de 2023, o objetivo passou a ser a compatibilidade binária com o RHEL, sem exigir que todos os componentes fossem cópias exatas. A documentação do projeto apresenta essa mudança como uma diferença entre a antiga abordagem 1:1 e a estratégia atual de compatibilidade (ALMALINUX WIKI, 2026).

O CentOS Stream possui outro posicionamento. Enquanto o AlmaLinux busca oferecer uma plataforma estável e compatível para aplicações Enterprise Linux, o CentOS Stream funciona como uma distribuição de desenvolvimento contínuo relacionada ao RHEL. Assim, apesar de fazerem parte do mesmo ecossistema, possuem objetivos diferentes.

Após a mudança de 2023, o AlmaLinux passou a utilizar fontes provenientes do ecossistema Enterprise Linux e outras fontes upstream para construir seus pacotes. O projeto também mantém seus próprios processos de construção, testes e distribuição (ALMALINUX WIKI, 2026).

A principal diferença entre reconstrução 1:1 e compatibilidade binária está no resultado esperado. Na primeira, busca-se reproduzir o sistema de maneira extremamente próxima. Na segunda, o objetivo é garantir que programas e

componentes compatíveis funcionem corretamente, mesmo que sua implementação interna não seja idêntica.

6. ABI E A EVOLUÇÃO DA ESTRATÉGIA DE COMPATIBILIDADE DO ALMALINUX

ABI significa *Application Binary Interface*, ou Interface Binária de Aplicação. Ela define como um programa já compilado se comunica com o sistema operacional, bibliotecas e outros componentes necessários para sua execução.

A estabilidade da ABI é importante em ambientes empresariais, pois aplicações podem permanecer em funcionamento durante muitos anos. Alterações incompatíveis podem exigir recompilação ou modificações na infraestrutura (RED HAT, 2019).

No AlmaLinux, a ABI ganhou importância a partir de 2023. Inicialmente, o projeto buscava oferecer uma reconstrução 1:1 do RHEL, reproduzindo seus componentes de maneira muito próxima. Porém, mudanças na disponibilização das fontes utilizadas para construir o RHEL dificultaram essa estratégia.

Diante disso, a AlmaLinux OS Foundation anunciou uma mudança. O projeto deixou de ter como objetivo principal a reconstrução 1:1 do RHEL e passou a priorizar a compatibilidade binária (ALMALINUX OS, 2023).

Isso significa que os componentes internos do AlmaLinux não precisam ser idênticos aos do RHEL, desde que as aplicações desenvolvidas para esse ambiente funcionem corretamente. Assim, sistemas com implementações diferentes podem manter compatibilidade quando oferecem as interfaces necessárias para a execução dos programas.

Essa mudança também permite maior liberdade no desenvolvimento do AlmaLinux, pois o projeto pode utilizar componentes diferentes quando necessário. O

foco passa a ser garantir a funcionalidade das aplicações e a estabilidade do sistema para seus usuários.

7. COMO FUNCIONA A COMPATIBILIDADE COM RHEL

A compatibilidade entre AlmaLinux e RHEL deve ser entendida principalmente a partir do comportamento das aplicações. Uma aplicação desenvolvida para RHEL depende de bibliotecas, interfaces e componentes do sistema. Quando essas interfaces são mantidas de forma compatível, o programa pode funcionar em outro sistema que ofereça as mesmas condições.

Um exemplo simplificado é:

Aplicação -> ABI -> Bibliotecas e componentes do sistema -> AlmaLinux/RHEL

A aplicação utiliza interfaces disponíveis no ambiente, enquanto a ABI estabelece as condições necessárias para que o programa compilado consiga interagir com elas. Se o AlmaLinux fornece uma implementação compatível, a aplicação pode funcionar sem modificações.

A Red Hat possui documentação específica sobre compatibilidade de aplicações e ABI. Essas diretrizes procuram reduzir problemas de migração e estabelecer interfaces que podem ser utilizadas pelas aplicações para preservar compatibilidade (RED HAT, 2019; RED HAT, 2025).

A compatibilidade também pode envolver módulos de kernel. O kernel possui interfaces e estruturas utilizadas por componentes específicos, tornando a compatibilidade de módulos relevante em ambientes que utilizam drivers ou componentes externos (RED HAT, 2024).

Outro ponto importante é que compatibilidade não significa código idêntico. Dois sistemas podem possuir implementações diferentes e ainda apresentar o comportamento necessário para executar uma aplicação.

Essa diferença é uma das principais consequências da mudança de estratégia do AlmaLinux. O projeto pode realizar modificações ou utilizar componentes diferentes quando necessário, desde que preserve o nível de compatibilidade esperado.

Para empresas, essa abordagem pode reduzir dificuldades na adoção de uma distribuição alternativa. Uma organização com aplicações desenvolvidas para RHEL pode avaliar o AlmaLinux sem exigir que todos os componentes internos sejam iguais.

Entretanto, compatibilidade não elimina a necessidade de testes. Uma aplicação pode depender de características específicas do ambiente ou de determinadas versões de bibliotecas. A documentação da Red Hat recomenda testes para situações específicas (RED HAT, 2014).

8. SEGURANÇA NO ALMALINUX

A segurança de um sistema operacional depende de diferentes mecanismos. No AlmaLinux, é necessário manter o sistema atualizado, controlar permissões, utilizar recursos de proteção e acompanhar alterações relevantes. As atualizações são distribuídas por meio de repositórios e erratas, que podem conter correções de segurança, bugs e melhorias.

Outro mecanismo importante é o SELinux, que utiliza políticas de segurança para controlar as ações que processos e usuários podem executar. No laboratório da disciplina, o SELinux deve permanecer em modo *Enforcing*, conforme os requisitos definidos no briefing.

Além desses mecanismos, a segurança depende da integridade dos componentes do sistema. Integridade de arquivos significa manter arquivos e configurações em um estado conhecido, permitindo identificar alterações que não deveriam ter ocorrido. Uma modificação pode ser legítima, como durante uma atualização, ou resultar de uma ação não autorizada.

Diretórios como */etc* possuem configurações importantes do sistema e de serviços, enquanto */bin*, */sbin* e */boot* contêm componentes relevantes para seu

funcionamento. As chaves SSH também são importantes por estarem relacionadas ao controle de acesso. Por esse motivo, esses elementos são considerados pelo *fim-check.sh*.

O objetivo do verificador não é impedir uma alteração, mas detectar mudanças comparando o estado atual com uma *baseline*, que representa o estado conhecido do sistema em determinado momento.

A partir dessa comparação, os arquivos podem ser classificados como **MODIFICADO**, quando seus conteúdos ou atributos são diferentes; **REMOVIDO**, quando estavam presentes na baseline, mas não estão mais no sistema; ou **NOVO**, quando aparecem no sistema sem terem sido registrados anteriormente.

Além do conteúdo, o *fim-check.sh* verifica proprietário, permissões e contexto SELinux. Dessa forma, a análise permite identificar diferentes tipos de alterações.

A integridade funciona, portanto, como uma camada complementar de segurança. Enquanto mecanismos como atualizações e SELinux ajudam a prevenir ou reduzir riscos, o verificador permite identificar alterações ocorridas no sistema. Os resultados devem ser analisados para determinar se a mudança foi legítima ou representa um possível problema de segurança.

9. INTEGRIDADE DE ARQUIVOS

Integridade de arquivos significa manter dados e componentes em um estado esperado, identificando alterações que não deveriam ter ocorrido. Em um sistema operacional, isso é importante porque determinados arquivos influenciam diretamente o funcionamento e a segurança da máquina.

Uma alteração pode ser legítima ou não. Quando o administrador instala uma atualização, por exemplo, determinados arquivos podem ser modificados de forma esperada. O problema ocorre quando um arquivo é alterado sem autorização ou explicação administrativa.

Arquivos presentes em */etc* podem conter configurações importantes do sistema e de serviços. Diretórios como */bin* e */sbin* podem conter executáveis

utilizados pelo sistema e pelos administradores, enquanto /boot contém componentes necessários à inicialização. As chaves SSH também são relevantes para o controle de acesso.

Por esse motivo, o fim-check.sh foi definido para monitorar esses locais. Seu objetivo não é impedir fisicamente alterações, mas registrar um estado conhecido e compará-lo com o estado atual.

A diferença entre prevenção e detecção é importante. Um controle preventivo busca impedir uma ação, enquanto um mecanismo de detecção procura identificar que algo aconteceu. O verificador de integridade pertence principalmente à segunda categoria.

A partir de uma baseline, podem ser registradas informações sobre os arquivos que deveriam existir e seus atributos. Em uma verificação posterior, essas informações são comparadas com o estado atual.

Se um arquivo possui conteúdo ou atributos diferentes, pode ser classificado como MODIFICADO. Se estava presente na baseline, mas não está mais no sistema, pode ser classificado como REMOVIDO. Se aparece sem estar registrado anteriormente, pode ser classificado como NOVO.

Além do conteúdo, o fim-check.sh também considera proprietário, permissões e contexto SELinux, tornando a verificação mais completa.

Esse mecanismo permite obter uma visão do estado dos componentes monitorados e fornece informações úteis para investigações de segurança.

10. SHA-256 E VERIFICAÇÃO DE INTEGRIDADE

Uma função hash recebe uma entrada, como o conteúdo de um arquivo, e produz um valor de tamanho definido chamado digest. Esse valor pode ser utilizado para comparar conteúdos e verificar se houve alteração.

O SHA-256 faz parte da família de algoritmos especificada pelo Secure Hash Standard. Para verificar integridade, o conteúdo de um arquivo é processado e gera

determinado digest. Se o conteúdo for alterado, o resultado da função hash também deverá ser diferente (NIST, 2015).

No fim-check.sh, esse mecanismo é utilizado para criar uma baseline. Ela representa o estado conhecido do sistema em determinado momento. Para cada arquivo monitorado, é armazenado o hash correspondente ao seu conteúdo.

O processo pode ser representado assim:

Arquivo Original -> SHA-256 -> Hash A

Posteriormente:

Arquivo Atual -> SHA-256 -> Hash B

Se Hash A = Hash B, o conteúdo permanece igual em relação à baseline.

Se Hash A for diferente de Hash B, o conteúdo foi alterado.

O NIST descreve os algoritmos da família Secure Hash Standard, incluindo o SHA-256, utilizados em operações criptográficas e verificações relacionadas à integridade (NIST, 2015).

Entretanto, o hash sozinho não resolve todo o problema. Se um atacante conseguir alterar o arquivo e também modificar a baseline utilizada na comparação, a alteração poderá não ser detectada. Por isso, a proteção da própria baseline é importante.

Além disso, uma diferença de hash indica que houve mudança no conteúdo, mas não determina quem realizou a alteração ou se ela foi autorizada. Para isso, é necessário analisar logs, atualizações e outras informações do sistema.

Assim, o fim-check.sh deve ser entendido como uma camada de detecção dentro de uma estratégia de segurança mais ampla.

11. VERIFICADOR DE INTEGRIDADE (fim-check.sh)

O fim-check.sh é o script desenvolvido para verificar a integridade de arquivos no AlmaLinux. Seu objetivo é comparar o estado atual de arquivos e atributos com uma baseline previamente registrada.

A primeira etapa consiste em verificar as condições necessárias para execução. O briefing determina o uso de set -euo pipefail, pelo menos cinco funções, suporte a -h e --help, verificação de privilégios por EUID e conferência de dependências com command -v.

Depois dessas verificações, o script pode carregar ou gerar a baseline, contendo as informações necessárias para comparar o estado conhecido com o atual.

Os locais monitorados incluem /etc, /bin, /sbin e /boot, além das chaves SSH. Para cada elemento, o script verifica o conteúdo utilizando SHA-256 e também considera proprietário, permissões e contexto SELinux, conforme os requisitos da atividade.

Durante a comparação, os resultados são classificados. Um arquivo presente na baseline com conteúdo ou atributos diferentes pode ser identificado como MODIFICADO. Um arquivo que deixou de existir pode ser classificado como REMOVIDO. Um arquivo encontrado atualmente e ausente da baseline pode ser identificado como NOVO.

O funcionamento pode ser resumido pelo fluxo:

Início -> Verificar privilégios -> Verificar dependências -> Carregar baseline -> analisar arquivos -> Comparar -> Classificar resultados -> Registrar Log -> Finalizar

A utilização de funções permite separar responsabilidades, como verificar dependências, registrar logs, calcular hashes, coletar atributos, comparar resultados e apresentar o relatório.

O logging também é obrigatório. O briefing determina registros com timestamp e níveis como INFO, AVISO e ERRO, armazenados em `/var/log/`. Também são definidos códigos de saída para indicar sucesso, achados, erro de uso e ausência de dependências.

O script deve ser idempotente, evitando alterações inesperadas quando executado repetidamente. Arquivos temporários devem ser criados com `mktemp` e removidos por meio de `trap`.

Também não devem existir senhas ou chaves secretas no código. O script deve passar pelo ShellCheck sem erros, e alterações de configuração devem possuir uma forma conhecida de reversão.

Dessa maneira, o `fim-check.sh` não é apenas um script que calcula hashes, mas uma solução estruturada de verificação de integridade, com controles de execução, identificação de alterações e registro de resultados.

12. AUTOMAÇÃO E COMPARAÇÃO COM AIDE

Uma verificação manual pode deixar alterações sem detecção por um período. A automação reduz essa dependência e permite verificações regulares.

No projeto, o `fim-check.sh` deve ser executado por um timer do `systemd`, que programa verificações periódicas e registra horários, arquivos analisados e alterações encontradas.

Os logs permitem identificar quando uma alteração ocorreu e analisar se ela foi legítima ou não.

Além do `fim-check.sh`, existem ferramentas de File Integrity Monitoring, como o AIDE, que verifica alterações com base em uma referência. Enquanto o AIDE é especializado, o `fim-check.sh` foi desenvolvido especificamente para os requisitos da atividade.

Característica	fim-check.sh	AIDE
Objetivo	Verificação de integridade	Verificação de integridade
Implementação	Script Shell próprio	Ferramenta especializada
SHA-256	Utilizado na baseline	Suportado por mecanismos de integridade
Diretórios	Definidos pelo projeto	Configuráveis
Dono e permissões	Verificados	Verificados conforme configuração
SELinux	Incluído nos requisitos	Depende da configuração
Automação	systemd timer	Pode ser automatizado
Complexidade	Menor	Maior
Finalidade no trabalho	Implementação própria	Solução especializada

13. ANÁLISE E APLICAÇÃO PRÁTICA

A mudança de estratégia do AlmaLinux demonstra que a compatibilidade não depende de possuir os mesmos componentes internos, mas de garantir o funcionamento das aplicações por meio da compatibilidade ABI (ALMALINUX OS, 2023).

Na segurança, a integridade funciona como uma camada adicional. Atualizações, repositórios e SELinux possuem funções importantes, enquanto o verificador identifica alterações nos arquivos.

O SHA-256 permite comparar o conteúdo atual com um estado conhecido. Por isso, o fim-check.sh também verifica proprietário, permissões e contexto SELinux.

A classificação entre Modificado, Removido e Novo facilita a interpretação dos resultados, indicando o tipo de alteração encontrada.

A automação por `systemd timer` melhora a solução ao realizar verificações regularmente, sem depender apenas da execução manual.

Uma limitação é que uma alteração não indica automaticamente uma ação maliciosa, pois atualizações legítimas também podem modificar arquivos. Por isso, os resultados devem ser analisados considerando o contexto do sistema.

Outra limitação é a proteção da própria baseline. Se um invasor conseguir alterar os arquivos monitorados e a base de comparação, a detecção poderá ser comprometida. Em um ambiente real, a baseline deve ser protegida adequadamente.

A comparação com o AIDE mostra que uma solução em Shell pode atender a necessidades específicas e educacionais, enquanto ferramentas especializadas oferecem recursos adicionais para ambientes de produção.

Como melhoria, o `fim-check.sh` poderia utilizar armazenamento mais seguro da baseline, relatórios detalhados, integração com sistemas de monitoramento e alertas automáticos, além de diferenciar mudanças esperadas de alterações inesperadas.

Dessa forma, a aplicação prática transforma conceitos de sistemas operacionais e segurança em uma solução funcional. O AlmaLinux fornece o ambiente Enterprise Linux, enquanto o `fim-check.sh` adiciona uma camada para identificar alterações na integridade dos arquivos.

14. CONCLUSÃO

O desenvolvimento deste trabalho permitiu analisar o AlmaLinux dentro do ecossistema Enterprise Linux, com foco na atuação da AlmaLinux OS Foundation e na mudança de estratégia ocorrida em 2023. O projeto surgiu como alternativa comunitária ao CentOS e inicialmente buscava uma reconstrução 1:1 do RHEL. Posteriormente, passou a priorizar a compatibilidade binária com o RHEL, concentrando seus esforços na execução de aplicações e workloads compatíveis (ALMALINUX OS, 2023).

A análise do conceito de ABI mostrou sua importância para ambientes empresariais. Aplicações compiladas dependem de interfaces de baixo nível, bibliotecas e outros componentes. Quando essas interfaces permanecem compatíveis, diferentes sistemas podem executar as mesmas aplicações mesmo que suas implementações internas não sejam idênticas (RED HAT, 2019).

Na parte de segurança, foi analisada a importância da integridade dos arquivos. O fim-check.sh foi definido para estabelecer uma baseline e verificar alterações em arquivos e diretórios importantes, utilizando SHA-256, proprietário, permissões e contexto SELinux. A solução classifica os resultados como Modificado, Removido e Novo, facilitando a identificação das alterações.

A utilização de um timer do systemd permite automatizar as verificações e reduzir a dependência da execução manual. A comparação com o AIDE mostrou que o script possui finalidade semelhante à de uma ferramenta especializada, porém com menor complexidade e foco nos requisitos definidos para o trabalho.

Portanto, o trabalho demonstra que a segurança de um sistema operacional depende de diferentes camadas. A compatibilidade permite que aplicações continuem funcionando, enquanto mecanismos de segurança e integridade ajudam a preservar a confiabilidade do ambiente.

16. REFERENCIAS

ALMALINUX OS. *A look back at 2023 for AlmaLinux.* [S. l.], 14 dez. 2023. Disponível em: <https://almalinux.org/blog/2023-12-14-2023-highlights/>. Acesso em: 15 set. 2026.

ALMALINUX OS. *The Future of AlmaLinux is Bright.* [S. l.], 13 jul. 2023. Disponível em: <https://almalinux.org/blog/future-of-almalinux/>. Acesso em: 15 set. 2026.

ALMALINUX OS FOUNDATION. *AlmaLinux OS Foundation.* [S. l.], 2026. Disponível em: <https://almalinux.org/foundation/>. Acesso em: 15 set. 2026.

ALMALINUX WIKI. *Errata (Security Advisories - ALSA, ALBA, ALEA).* [S. l.], 2026. Disponível em: <https://wiki.almalinux.org/documentation/errata.html>. Acesso em: 15 set. 2026.

ALMALINUX WIKI. *Repositories.* [S. l.], 2026. Disponível em: <https://wiki.almalinux.org/repos/>. Acesso em: 15 set. 2026.

ANDERSON, Tim. *Island in the Stream: AlmaLinux project issues first stable release of CentOS replacement.* The Register, 30 mar. 2021. Disponível em: <https://www.theregister.com/software/2021/03/30/island-in-the-stream-almalinux-project-issues-first-stable-release-of-centos-replacement/>. Acesso em: 15 set. 2026.

NIST - NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *FIPS PUB 180-4: Secure Hash Standard (SHS).* Gaithersburg, MD: U.S. Department of Commerce, ago. 2015. Disponível em: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>. Acesso em: 15 set. 2026.

RED HAT. *Red Hat Enterprise Linux 7: Application Compatibility Guide.* [S. l.], 2014. Disponível em: <https://access.redhat.com/articles/rhel-abi-compatibility>. Acesso em: 15 set. 2026.

RED HAT. *Red Hat Enterprise Linux 8: Application Compatibility Guide.* [S. l.], 2019. Disponível em: <https://access.redhat.com/articles/rhel8-abi-compatibility>. Acesso em: 15 set. 2026.

RED HAT. *Creating kernel modules for Red Hat Enterprise Linux*. [S. l.], 2024.
Disponível em: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/.
Acesso em: 15 set. 2026.

RED HAT. *Red Hat Enterprise Linux 9: Application Compatibility Guide*. [S. l.], 2025.
Disponível em: <https://access.redhat.com/articles/rhel9-abi-compatibility>. Acesso em:
15 set. 2026.